Embedded Controller

Timer / Timer Interrupt / PWM

2024

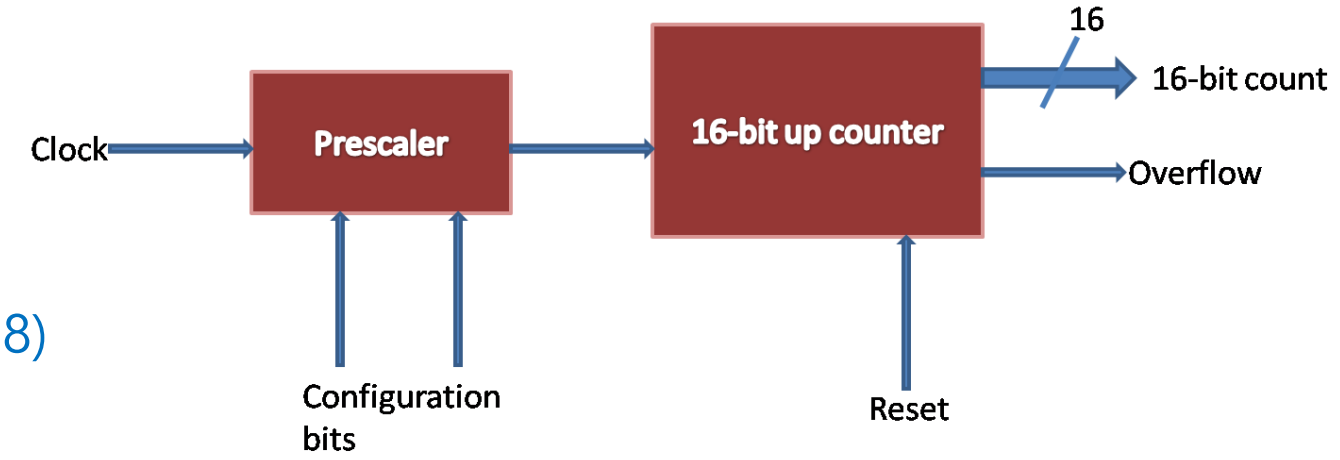
Young-Keun Kim
School of Mechanical & Control Engineering

Handong Global University

Timer

Timer

- **Timer**
  - Count increment or decrement in every CLK cycle
  - A hardware component
    - independent to CPU resource
  - Each timer is independent
    - Cortex-M3/4 has 32-bit, 16-bit timers
    - 11 timers (STM32F411)
    - 8 timers (STM32F411RE, excludes TIM 6,7,8)
    - each timer has ~4 channels

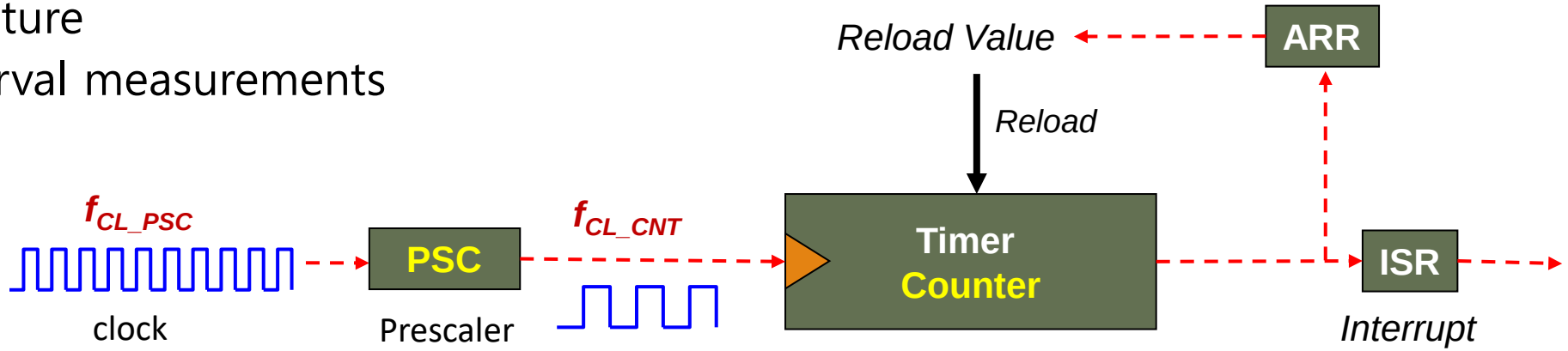


  - Timer is NOT SysTick
    - Timer Interrupt is different from SysTick Interrupt
    - Timer(Peripheral), SysTick (Processor Core)

한동대학교 HANDONG GLOBAL UNIVERSITY

Timer

- **Timer**
  - Composed of: (1) Prescaler (2) Auto-reload Counter
  - All timers are independent.
    - Can control different frequencies of each timer
  - Applications
    - Timer Interrupt
    - Output Compare
    - PWM
    - Input Capture
    - Time-interval measurements

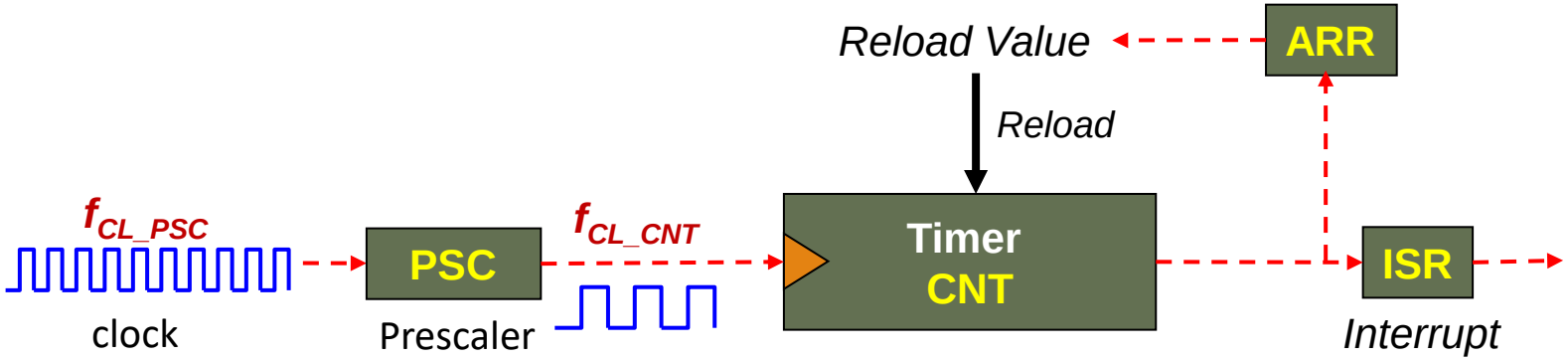


Timer CLK: $f_{CL_CNT} = \frac{f_{CL_PSC}}{PSC + 1}$

Timer

- **Counter**
  - Counting modes: 3
    - UP / DOWN / CENTER-ALIGN
  - Update events: 2
    - Overflow and Underflow

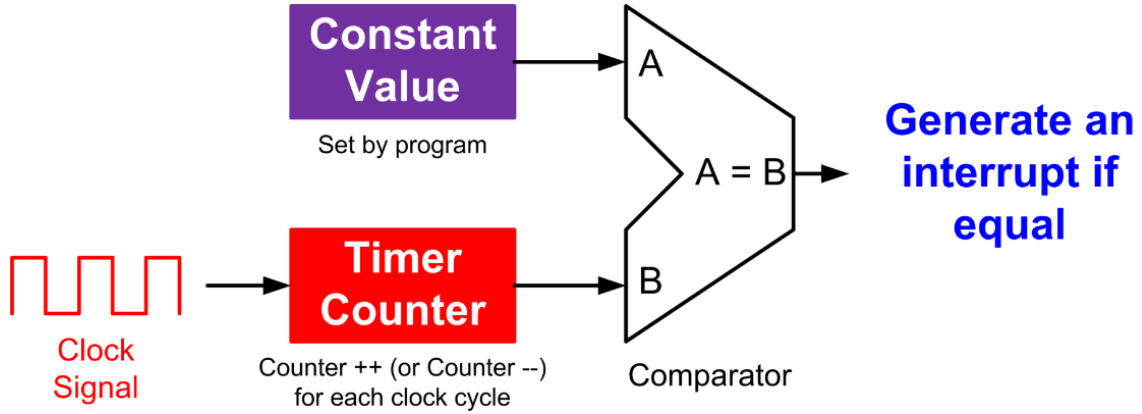
- **Timer register**
  - Prescaler Register (TIMx_PSC)
  - Counter Register (TIMx_CNT)
  - Auto-Reload Register (TIMx_ARR)



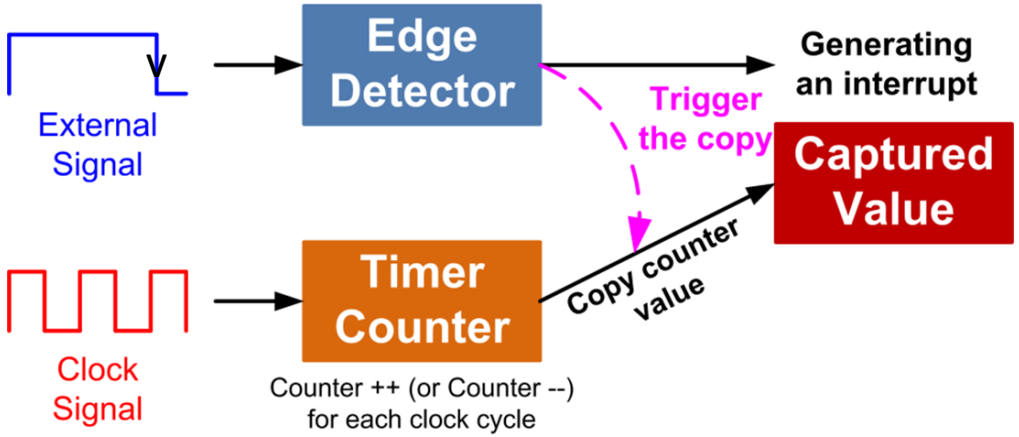
$$f_{CL_CNT} = \frac{f_{CL_PSC}}{PSC + 1}$$

Timer

- **Output Compare**
  - Generates output or interrupt by using comparator
  - Timer counter with programmed constant value



- **Input Capture**
  - Acquire counter value when edge of external signal is detected



한동대학교 HANDONG GLOBAL UNIVERSITY

Timer

- **Timer in STM32f4**
  1) Advanced-control timers (TIM1, TIM8*)
  2) General purpose timers (32-bit: TIM 2~5), (16-bit: TIM 9~14)
  3) Basic timers* (TIM 6~7)

Arduino Uno
- 8-bit Timer (TIM0, TIM2)
- 16-bit Timer (TIM1)

*STM32F411RE: TIM1, TIM2~5, 9~11

Timer type	Timer	Counter resolution	Counter type	Prescaler factor	DMA request generation	Capture/ compare channels	Complementary output
Advanced-control	TIM1, TIM8	16-bit	Up, Down, Up/down	Any integer between 1 and 65536	Yes	4	Yes
General purpose	TIM2, TIM5	32-bit	Up, Down, Up/down	Any integer between 1 and 65536	Yes	4	No
	TIM3, TIM4	16-bit	Up, Down, Up/down	Any integer between 1 and 65536	Yes	4	No
	TIM9	16-bit	Up	Any integer between 1 and 65536	No	2	No
	TIM10, TIM11	16-bit	Up	Any integer between 1 and 65536	No	1	No
	TIM12	16-bit	Up	Any integer between 1 and 65536	No	2	No
	TIM13, TIM14	16-bit	Up	Any integer between 1 and 65536	No	1	No
Basic	TIM6, TIM7	16-bit	Up	Any integer between 1 and 65536	Yes	0	No

STM32F411RE:
- TIM1
- TIM2~5
- TIM9~11

[Reference Manual Ch12~14]

한동대학교 HANDONG GLOBAL UNIVERSITY

Timer

- **Timer in STM32f4**

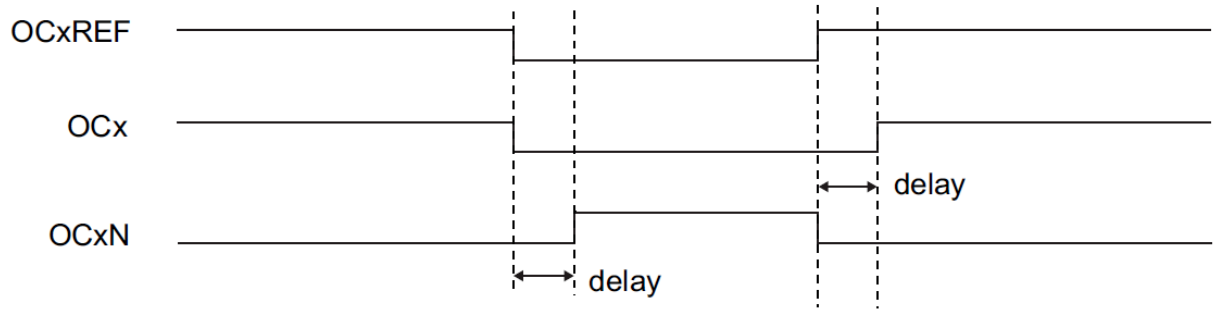
  1) Advanced-control timers (TIM1 & TIM8*)

  *STM32F411RE has no TIM8

  - 16-bit auto-reload counter
  - Up to 4 independent channels (TIMx_CH1 ~ TIMx_CH4)
  - Generate **complementary** PWM with programmable dead-time
  - Independent to general-purpose timers (TIM2~)

Timer type	Timer	Counter resolution	Counter type	Prescaler factor	DMA request generation	Capture/ compare channels	Complementary output
Advanced-control	TIM1, TIM8	16-bit	Up, Down, Up/down	Any integer between 1 and 65536	Yes	4	Yes

Figure 73. Complementary output with dead-time insertion.

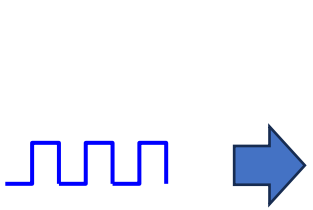


The dead time is necessary to prevent the **short circuit** of the power supply in pulse width modulated (PWM) voltage inverters results in output voltage deviations.

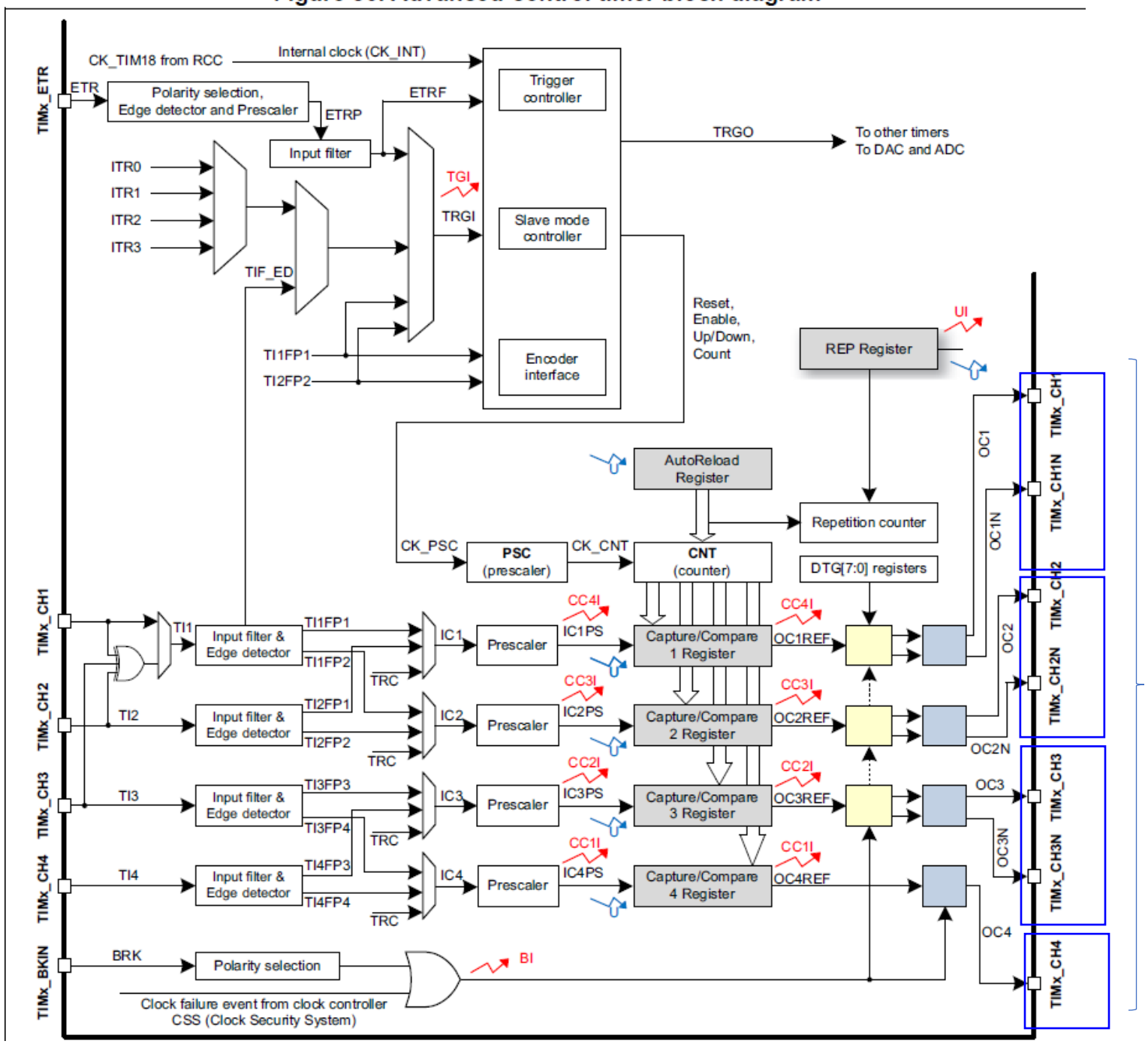
한동대학교 HANDONG GLOBAL UNIVERSITY

(TIM1 & TIM8)

4 independent channels (TIMx_CH1 ~ TIMx_CH4)
x=1 or 8



Input capture





Output Compare

Timer

- **Timer in STM32f4**

2) General-purpose timer (TIM2~TIM5)

- 32-bit (TIM2 and TIM5)
- 16-bit (TIM3 and TIM4)
- Up to 4 independent channels each Timer

2) General-purpose timer (TIM9~TIM11)

- 16-bit Up-counter (TIM9~11)
- Up to 2 independent channels each Timer

3) Basic Timer (TIM6 and TIM7)*

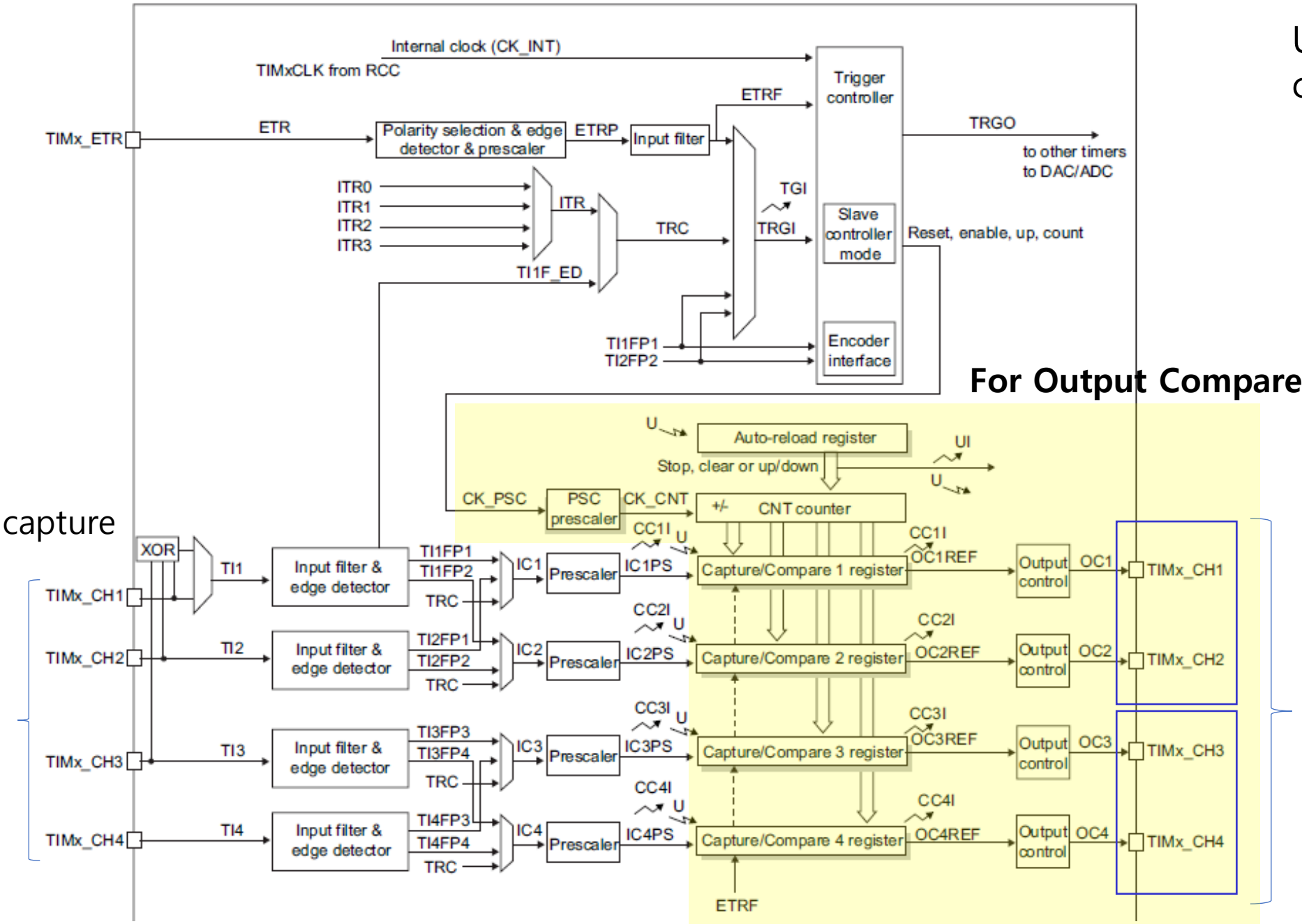
- 16-bit up count only, no capture
- Usually used for driving DAC

Timer type	Timer	Counter resolution	Counter type	Prescaler factor	DMA request generation	Capture/ compare channels	Complementary output
Advanced-control	TIM1, TIM8	16-bit	Up, Down, Up/down	Any integer between 1 and 65536	Yes	4	Yes
General purpose	TIM2, TIM5	32-bit	Up, Down, Up/down	Any integer between 1 and 65536	Yes	4	No
	TIM3, TIM4	16-bit	Up, Down, Up/down	Any integer between 1 and 65536	Yes	4	No
	TIM9	16-bit	Up	Any integer between 1 and 65536	No	2	No
	TIM10, TIM11	16-bit	Up	Any integer between 1 and 65536	No	1	No
	TIM12	16-bit	Up	Any integer between 1 and 65536	No	2	No
	TIM13, TIM14	16-bit	Up	Any integer between 1 and 65536	No	1	No
Basic	TIM6, TIM7	16-bit	Up	Any integer between 1 and 65536	Yes	0	No

한동대학교 HANDONG GLOBAL UNIVERSITY

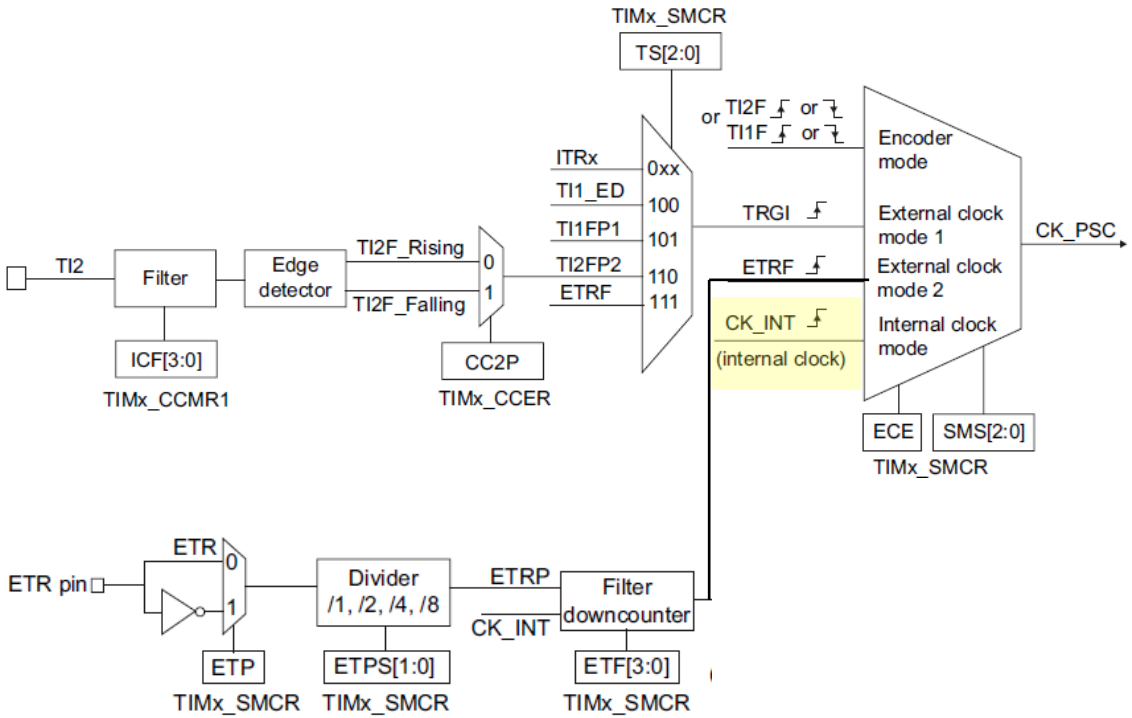
(TIM2~TIM5)
Up to 4 independent channels each Timer

Figure 134. General-purpose timer block diagram



Timer Clock

- **Selecting Clock for Timer Counter**
  - -Internal Clock (CK_INT) : default
  - External Clock mode 1: External Input Pin (TRGI) TIx
  - External Clock mode 2: External Trigger Input (ETR)
  - Internal trigger(ITRx): from other timer output, x=0~3

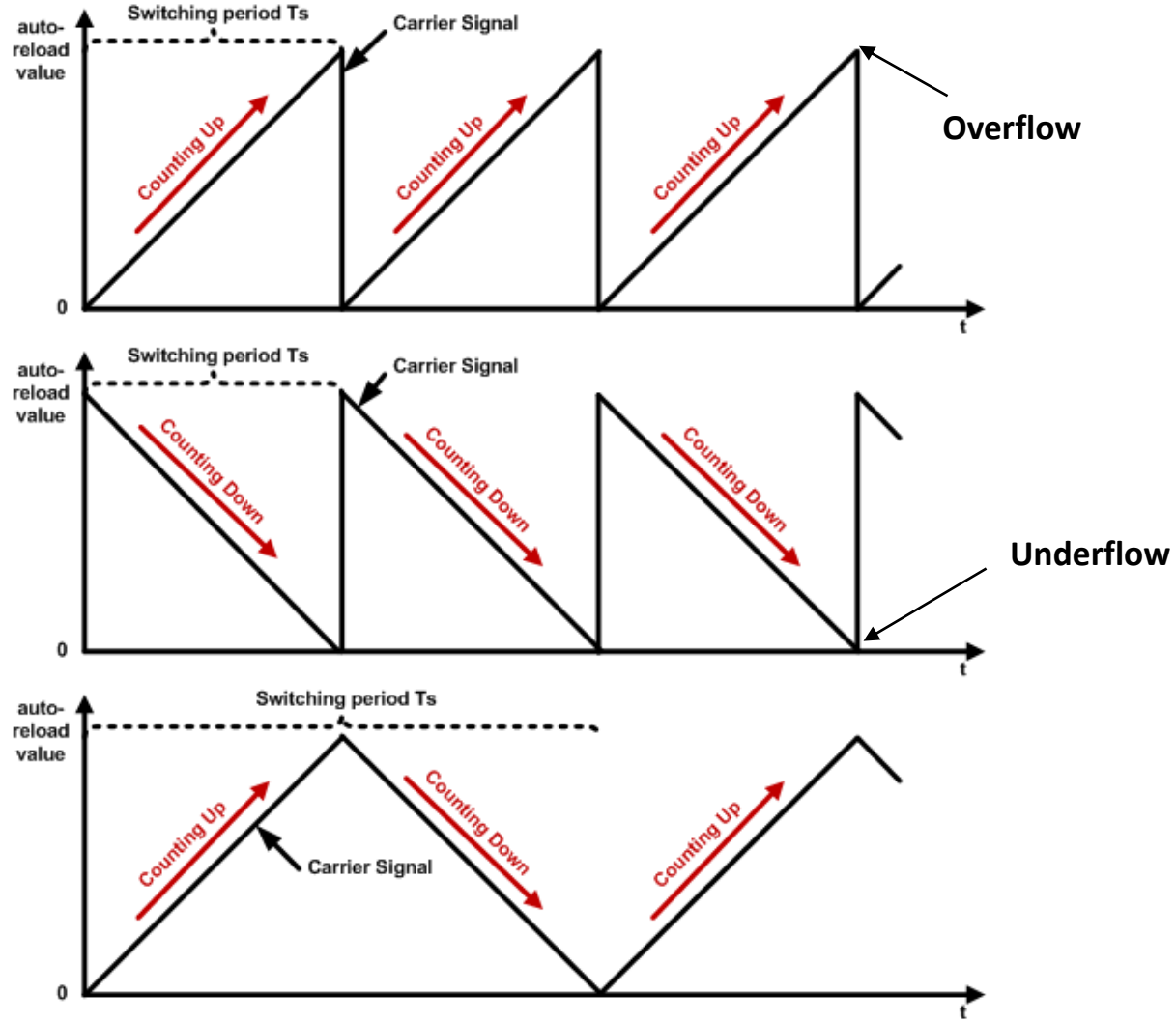


[RM Fig.61]

한동대학교 HANDONG GLOBAL UNIVERSITY

Counter Mode

- **Counting modes**
  : UP/DOWN/CENTER-ALIGN



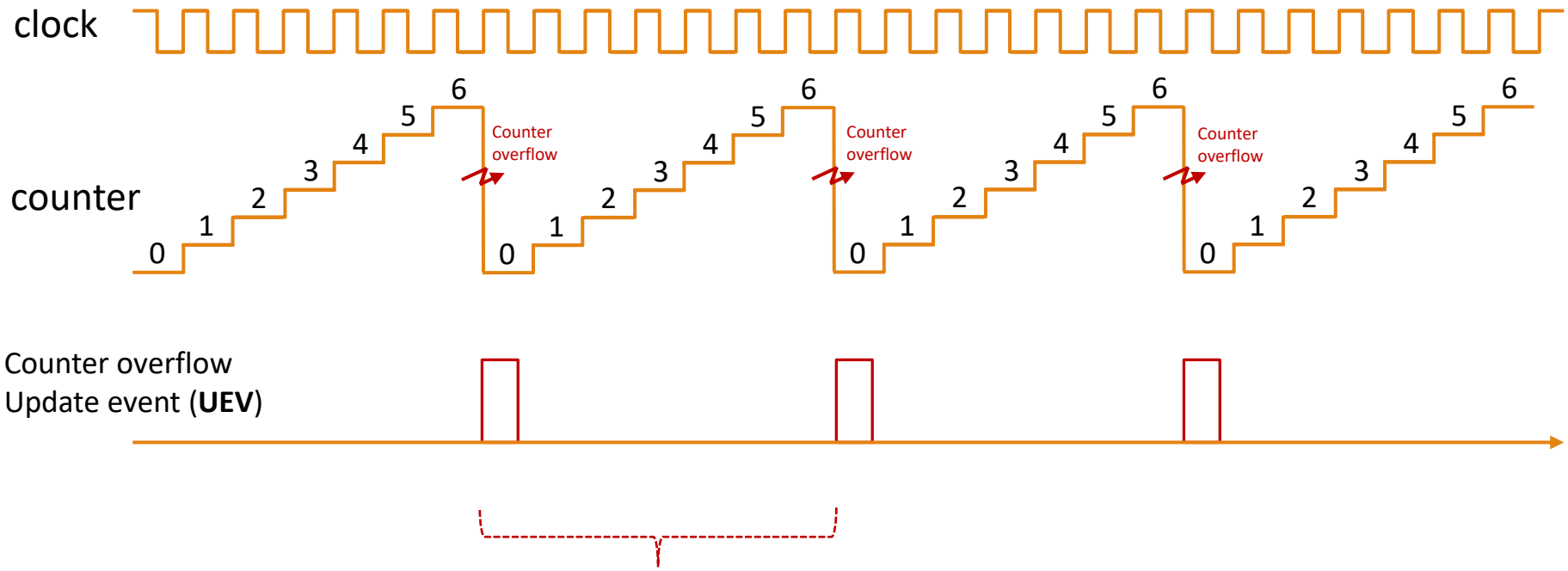
$$\text{Count Period} = (1 + ARR)\frac{1}{f_{CK_CNT}}$$

$$\text{Count Period} = (2ARR)\frac{1}{f_{CK_CNT}}$$

한동대학교 HANDONG GLOBAL UNIVERSITY

Counter Mode: Up-counting

Example: up counting with ARR = 6

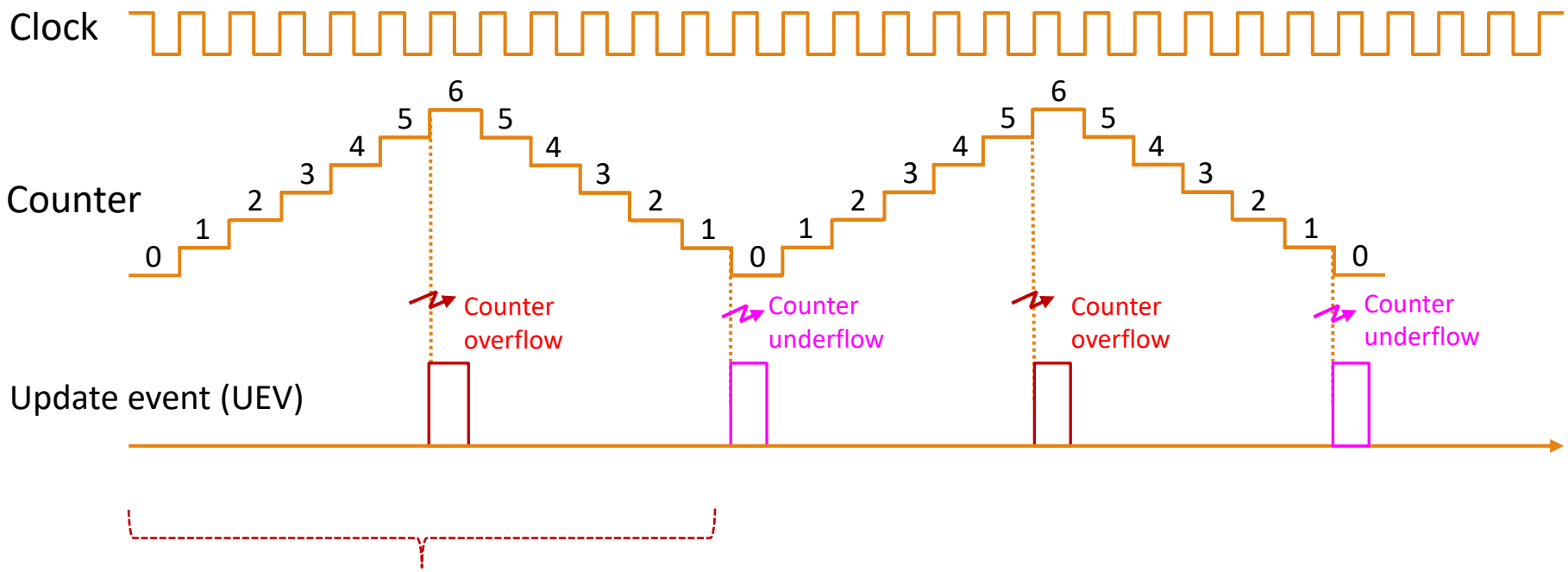


UEV Period = (1 + ARR) * Clock Period
= 7 * Clock Period

한동대학교 HANDONG GLOBAL UNIVERSITY

Counter Mode: Center-aligned Mode

Example: Center-aligned counting with ARR = 6



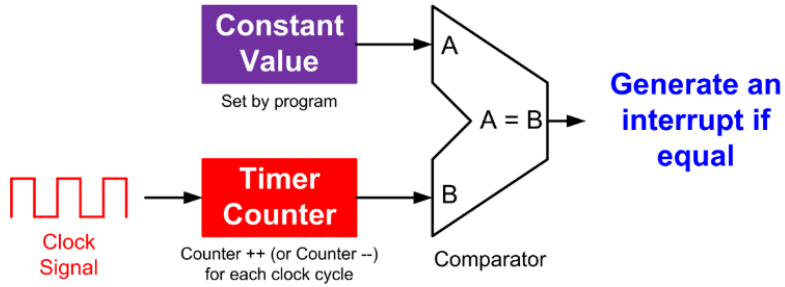
Period = 2 * ARR * Clock Period
= 12 * Clock Period

한동대학교 HANDONG GLOBAL UNIVERSITY

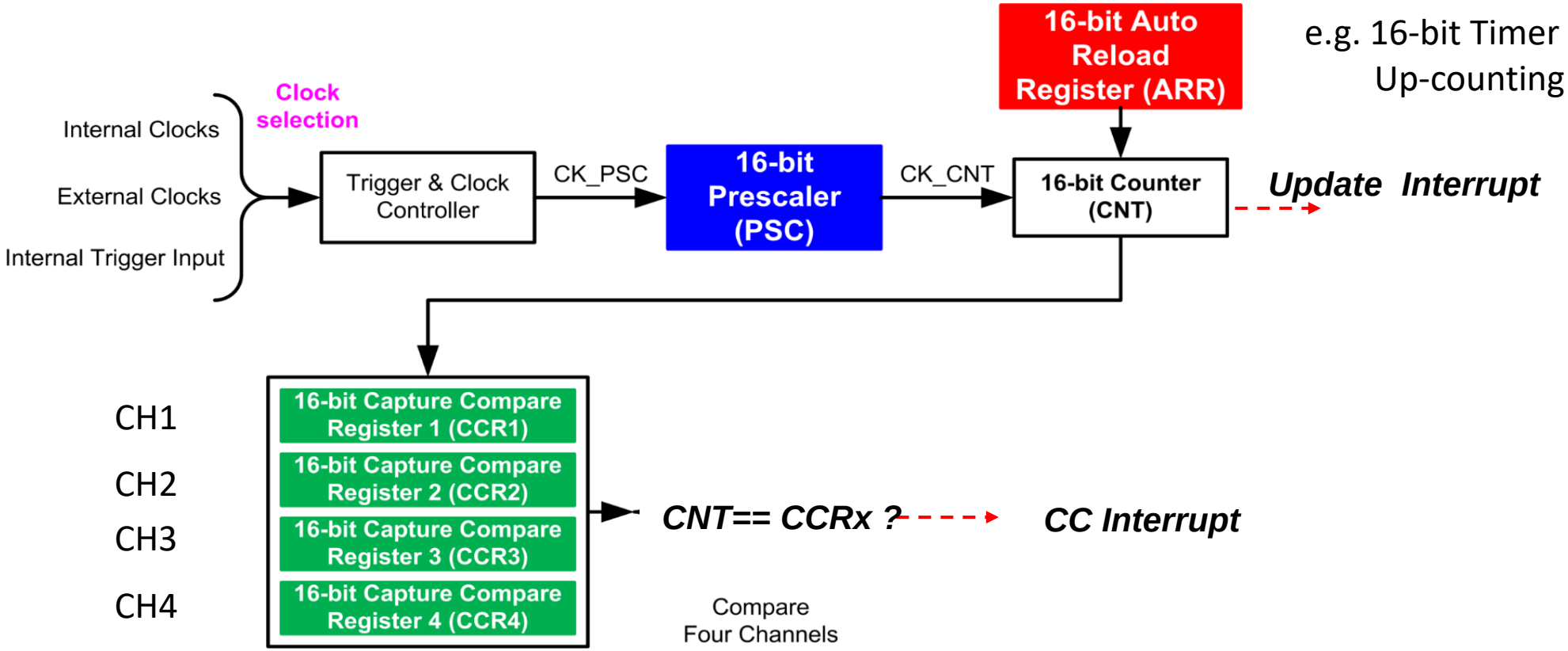
Timer Interrupt
(No Output)

Timer: Timer Interrupt

- **Application 1: Timer Interrupt**
  - Configured to generate an output signal in every periodic time
  - Control Loop, Acquisition Loop



e.g. 16-bit Timer
Up-counting



한동대학교 HANDONG GLOBAL UNIVERSITY

Timer: Timer Interrupt

- **Timer Interrupt :**
  - Different from SysTick Interrupt
  - Timer Interrupts
    (1) **U**pdate **EV**ent Interrupt: CNT has overflow or underflow
    (2) **C**ompare & **C**apture Interrupt: CNT matches CCR

  - Which event has occurred?
    - Read flag in Status Register (TIMx_SR)
      (1) **UIF** (**U**pdate **I**nterrupt **F**lag): Overflow or Underflow event
      (2) **CCIF**(**C**apture & **C**ompare **I**nterrupt **F**lag): CNT==CCR event

```
void  TIMx_IRQHandler(void){
   //  Check which interrupt
   //  if (is_CCIF(TIMx) ) ?    if (is_UIF (TIMx) ) ?

}
```


한동대학교 HANDONG GLOBAL UNIVERSITY

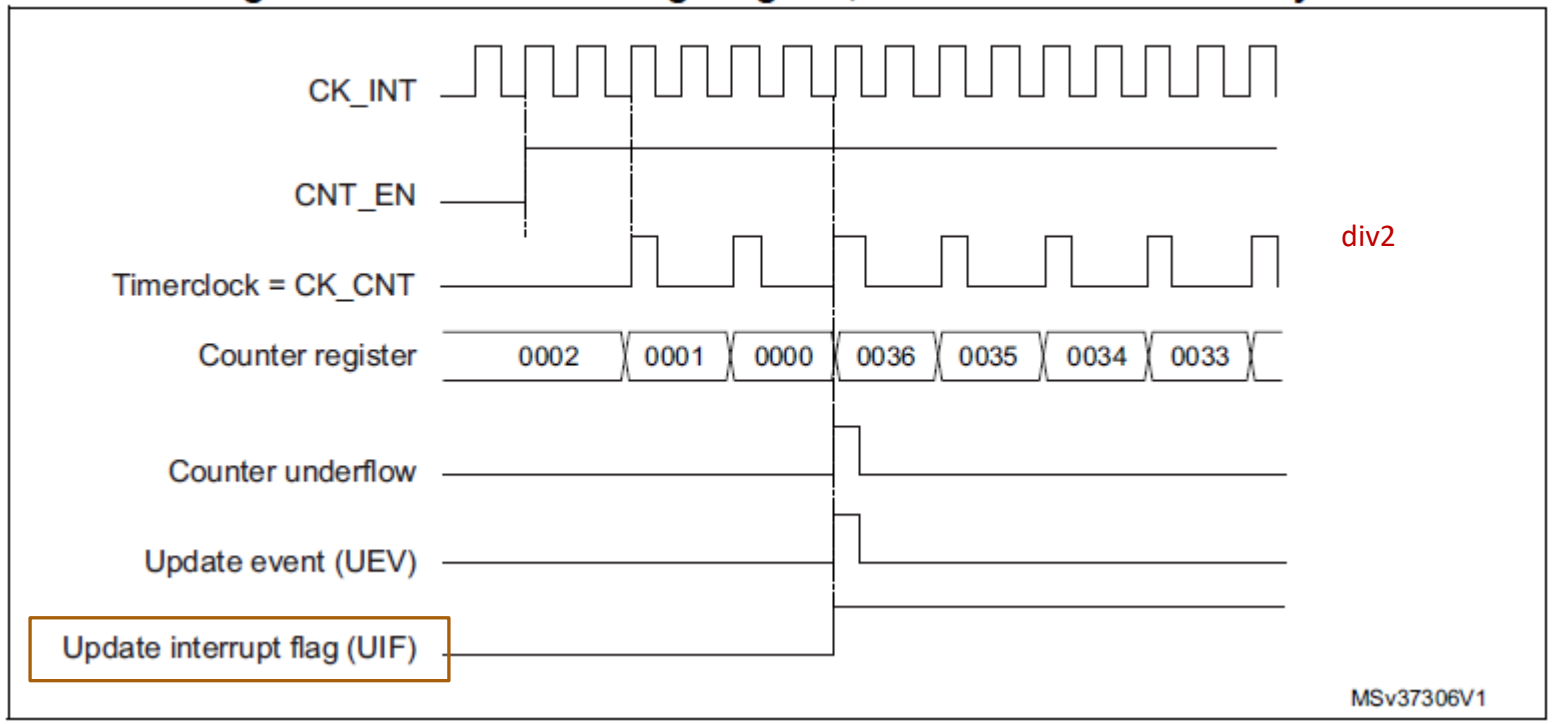
Timer: Timer Interrupt

(1) CNT Update Interrupt flag (UIF): overflow or underflow

Example: Down-counting with TIMx_ARR=0x36 , TIMx_PSC=2

UEV Period = (1 + ARR) * CK_CNT_Period

Figure 144. Counter timing diagram, internal clock divided by 2

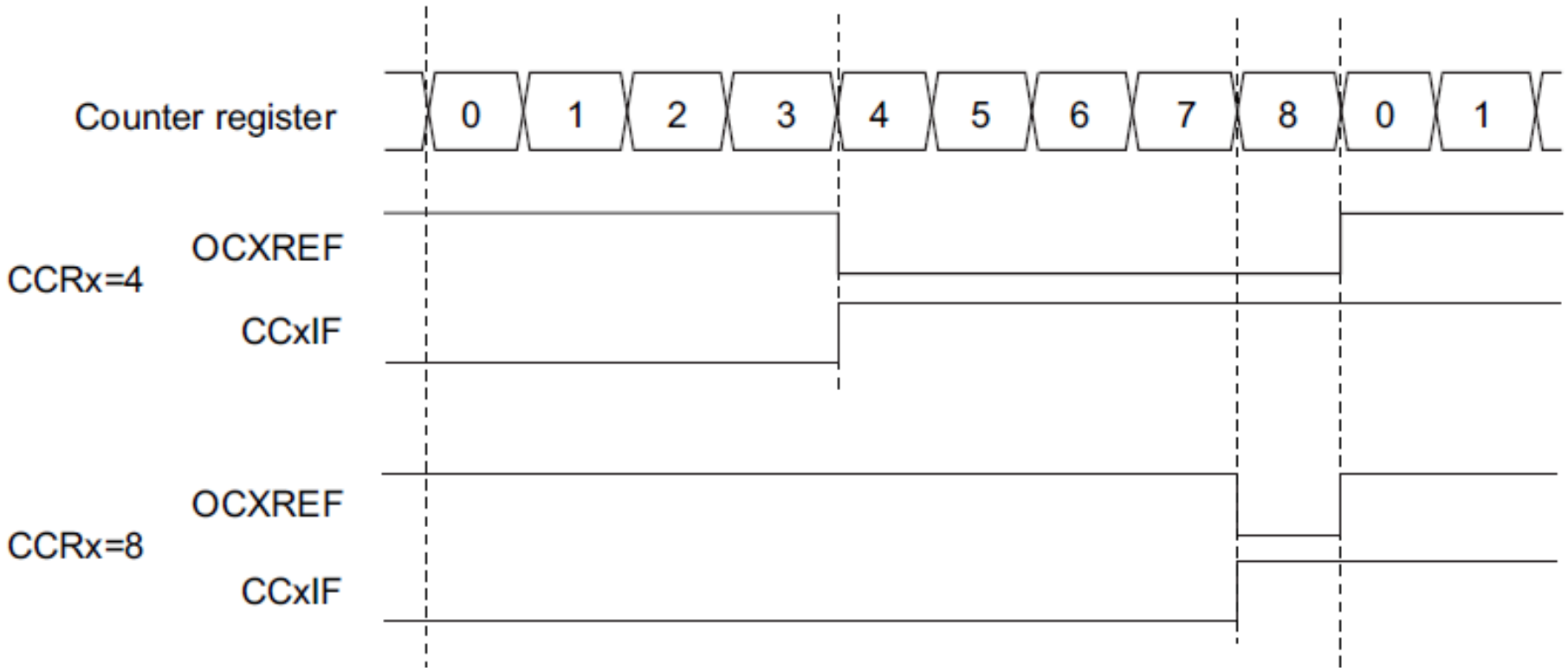


div2

한동대학교 HANDONG GLOBAL UNIVERSITY

Timer: Timer Interrupt

(2) Compare-Capture_x Interrupt Flag(CCxIF) : triggers when CNT==CCRx, (**see PWM)**



OCxREF → OCx PWM pulses

Interrupt flag

한동대학교 HANDONG GLOBAL UNIVERSITY

Timer Interrupt

● TIMx Setting and TIMx Interrupt (no output mode)

System Clock setting

1. RCC setting (HSI/HSE/PLL)

Timer setting

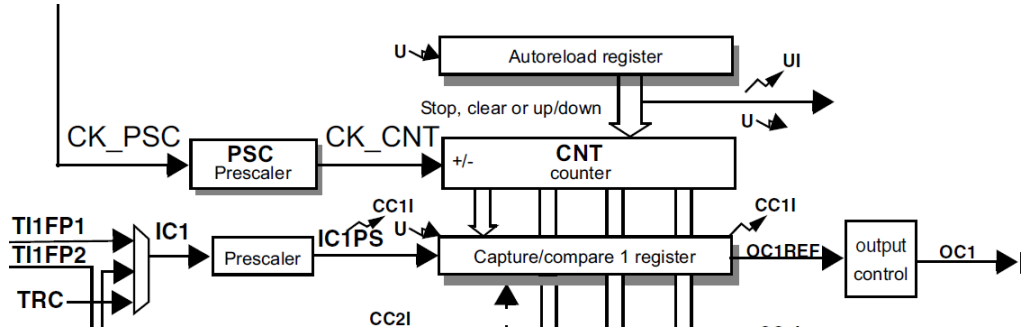
1. Enable Timer Peripheral Clock	(**RCC → APB1ENR**)
2. Set Counting Direction	(**TIMx → CR1 : DIR**)
3. Set Timer Clock Pre-scaler value	(**TIMx → PSC : PSC[15:0]**)
4. Set Auto-reload value	(**TIMx → ARR : ARR**)
5. Enable Update or Compare Interrupt	**(TIMx → DIER : UIE)** or **(: CCyE)**
6. Enable counter	(**TIMx → CR1:CEN**)

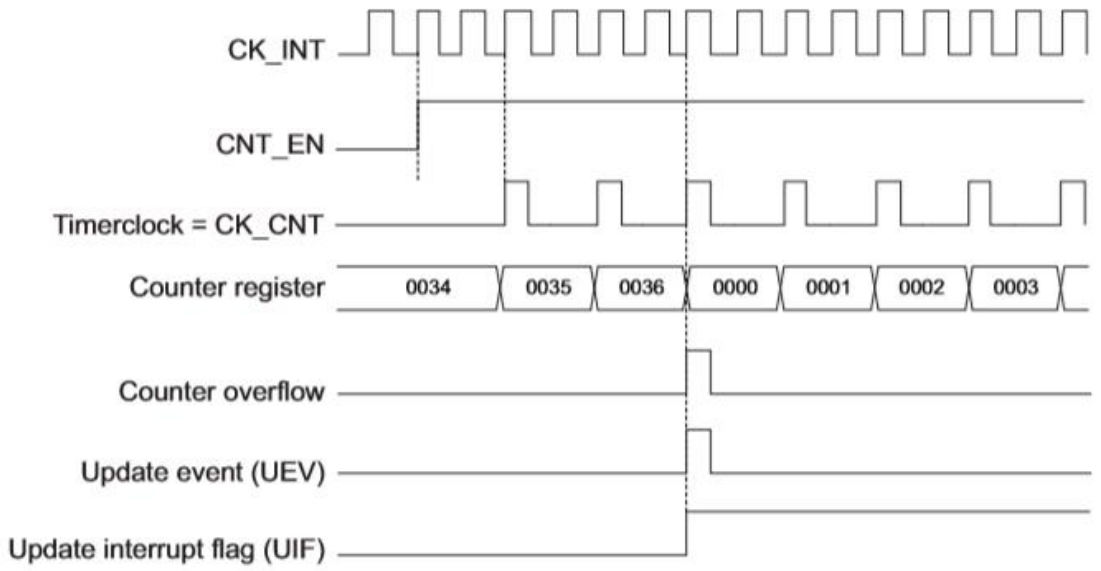
NVIC setting

1. Set interrupt Priority	NVIC_SetPriority(TIMx_IRQn,2)
2. Enable TIMx Interrupt:	NVIC_EnableIRQ(TIMx_IRQn)

*Usually use G.P. Timer

*Usually S.P. Timer (TIM1) is for PWM output mode

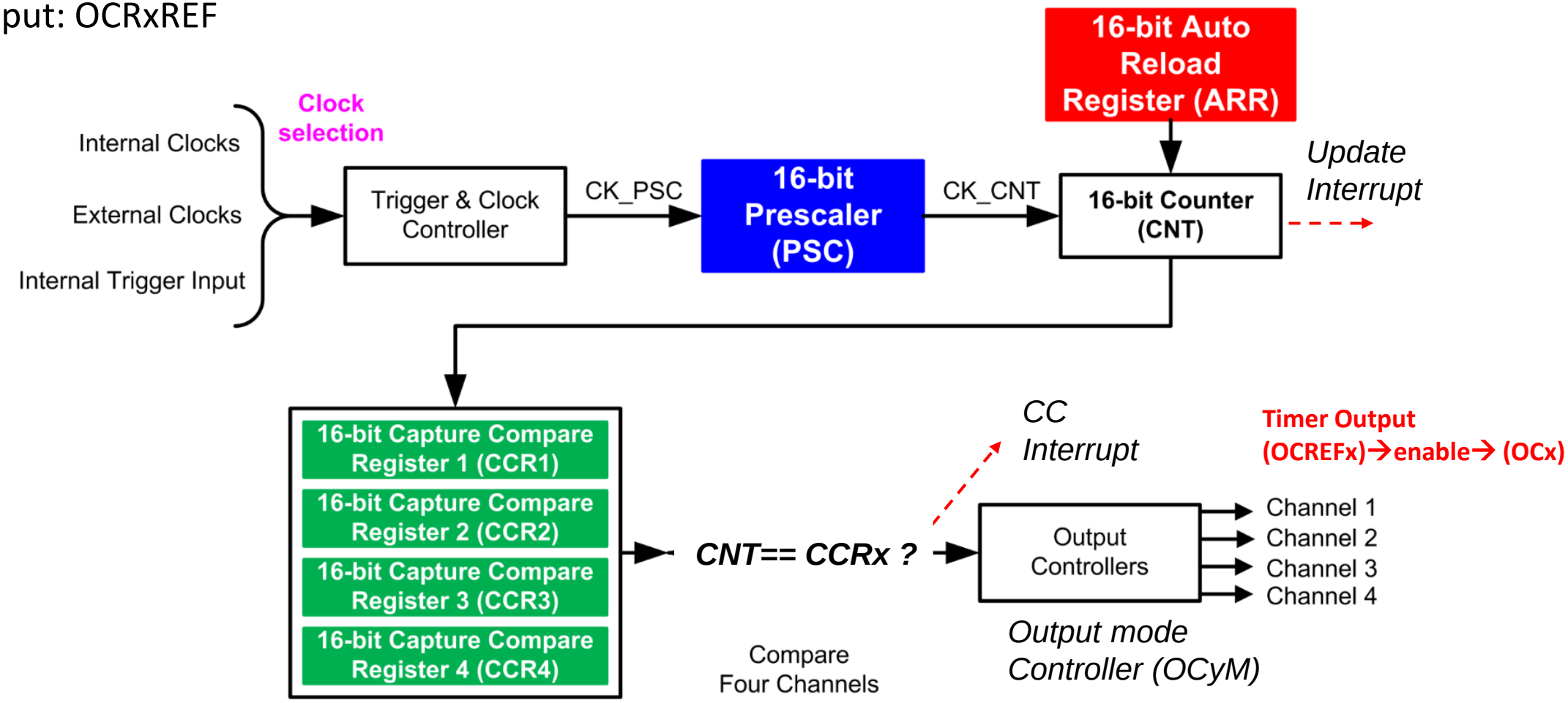




Output Compare (Output mode)

Timer: Output Compare

- **Application 2: Output compare and Timer Interrupt**
  - Timer Output by comparing CNT with Compare & Capture (CCR)
  - Output: OCRxREF



한동대학교 HANDONG GLOBAL UNIVERSITY

Timer: Output Compare

- **Setting Output Mode for channels (OCyM)**

  When CNT is compared to CCR, we can choose compare mode

  - Output Compare Mode (**OCyM**) in **TIMx_CCMR**. y=1 to 4 (channel)

Output Compare Mode (OCyM)	Timer Reference Output (OCyREF)
000	Frozen
001	H if CNT == CCR
010	L if CNT == CCR
011	Toggle if CNT == CCR
100	Forced low (always low)
101	Forced high (always high)
110	PWM mode 1
111	PWM mode 2

한동대학교 HANDONG GLOBAL UNIVERSITY

Timer: Output Compare

Register Setting

GPIO Pin setting

1. Set RCC
2. AF(TIMx) mode selection for Pin_y in GPIOn

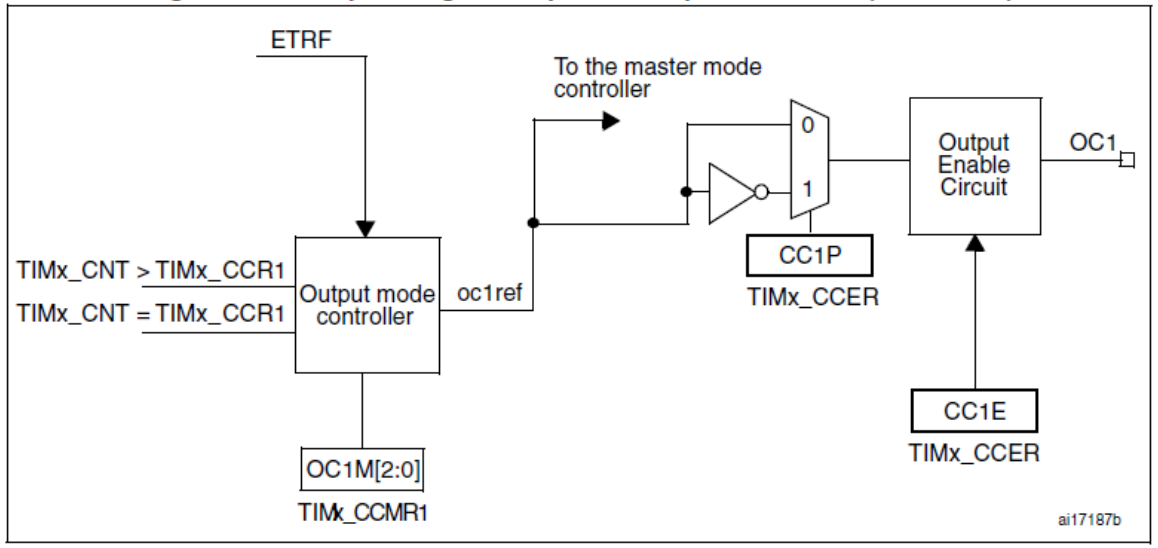
Timer setting:

1. Enable Timer peripheral Clock (**RCC → APB1ENR**)
2. Set Counting direction (**TIMx → CR1:DIR**)
3. Set Timer clock Pre-scaler value (**TIMx → PSC : PSC[15:0]**)
4. Set Auto-reload value (**TIMx → ARR : ARR**)
5. Enable Counter (**TIMx → CR1 : EN**)

Compare Output setting:

1. Select Output mode (**TIMx → CCMR : OCyM**)
2. Set CompareCapture value (**TIMx → CCRy : CCR**)
3. Select Output Polarity (**TIMx → CCER : CCyP**)
4. Enable CompareCaptureOutput (**TIMx → CCER : CCyE**)

Figure 114. Output stage of capture/compare channel (channel 1)

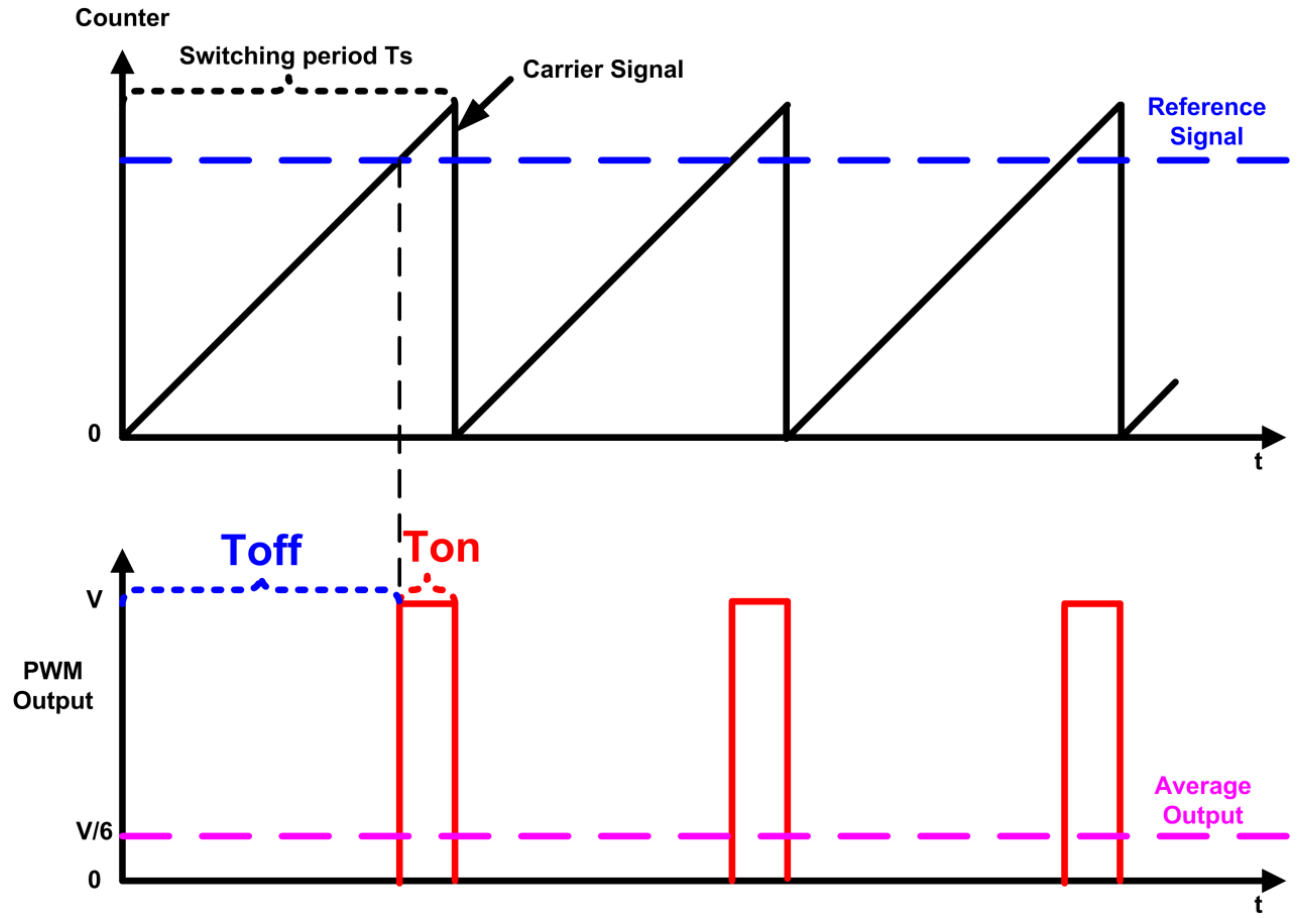


*default internal clock source CK_INT

PWM
(Special case of Output Compare)

Timer Application: PWM

- **Pulse Width Modulation(PWM)**
  - Special Type of Output Compare mode: OCyM=110, 111 (PWM mode1, mode2)
  - Rectangular waveform with duty ratio and period
  - Used for motor control, DC-to-DC power conversion, audio etc



$$\text{Duty Cycle} = \frac{T_{on}}{T_{on} + T_{off}}$$

한동대학교 HANDONG GLOBAL UNIVERSITY

Timer Application: PWM

Generating PWM

- Comparing **CNT?== CCR**
- Select **output mode(OCyM) as PWM1 or PWM2**
- Set Polarity bit
- TIM_x has up to 4 channels: same timer counter period
  - But different CCR→ different duty ratio

e.g. TIM2: up to 4 PWMs of same frequency

PWM output mode: (TIMx_CCMR →OCyM), y=1 to 4 (channel)

Mode	OCyM	CNT < CCR	CNT ≥ CCR
PWM mode 1 (Low True)	110	H(Active)	L(Inactive)
PWM mode 2 (High True)	111	L	H

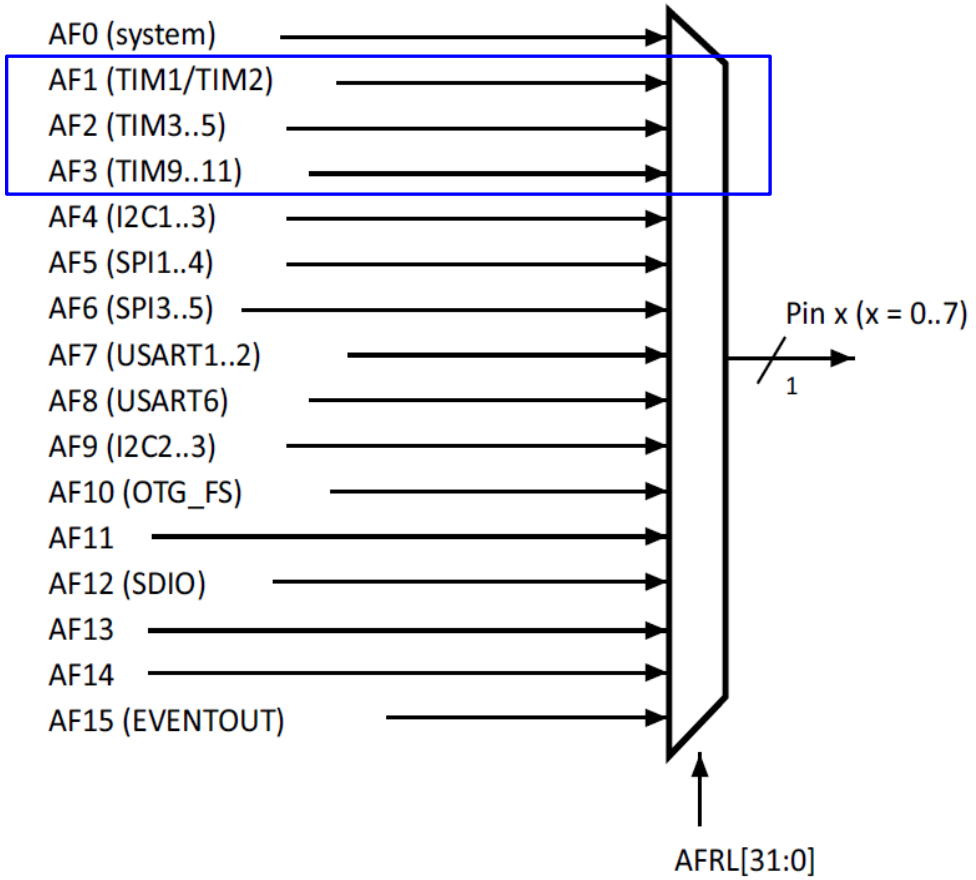
Output Polarity: (TIMx_CCER→ CCyP)

	Active	Inactive
Active High	High Voltage	Low Voltage
Active Low	Low Voltage	High Voltage

한동대학교 HANDONG GLOBAL UNIVERSITY

Timer Output Pin

- **Select Pin for Timer Output**
  - AF (Alternative Function) mode for GPIOx
    - For Pin x (x=0 to 7) use **AFRL** in GPIO register to select timer output
    - For Pin x (x=8 to 15) use **AFRH** in GPIO register to select timer output



PA0 // TIM2_Ch1
AFRL[3:0]= b0001 // AFRL0[3:0]

PA1 //TIM2_Ch2
AFRL[7:4]= b0001 // AFRL1[3:0]

…

PA8 // TIM1_Ch1
AFRH[3:0]= b0001 // AFRH8[3:0]

See Timer pinmap

한동대학교 HANDONG GLOBAL UNIVERSITY

Timer Application : PWM

● PWM registers

GPIO Pin setting

1. Set RCC for GPIO
2. AF(TIMx) mode selection for Pin_y in GPIO_n

Timer setting:

1. Enable Timer peripheral Clock	(**RCC → APB1ENR**)
2. Set Counting direction	(**TIMx → CR1 : DIR**)
3. Set Timer clock Pre-scaler value	(**TIMx → PSC : PSC[15:0]**)
4. Set Auto-reload value	(**TIMx → ARR : ARR**)
5. Enable Counter	(**TIMx → CR1 : CEN**)

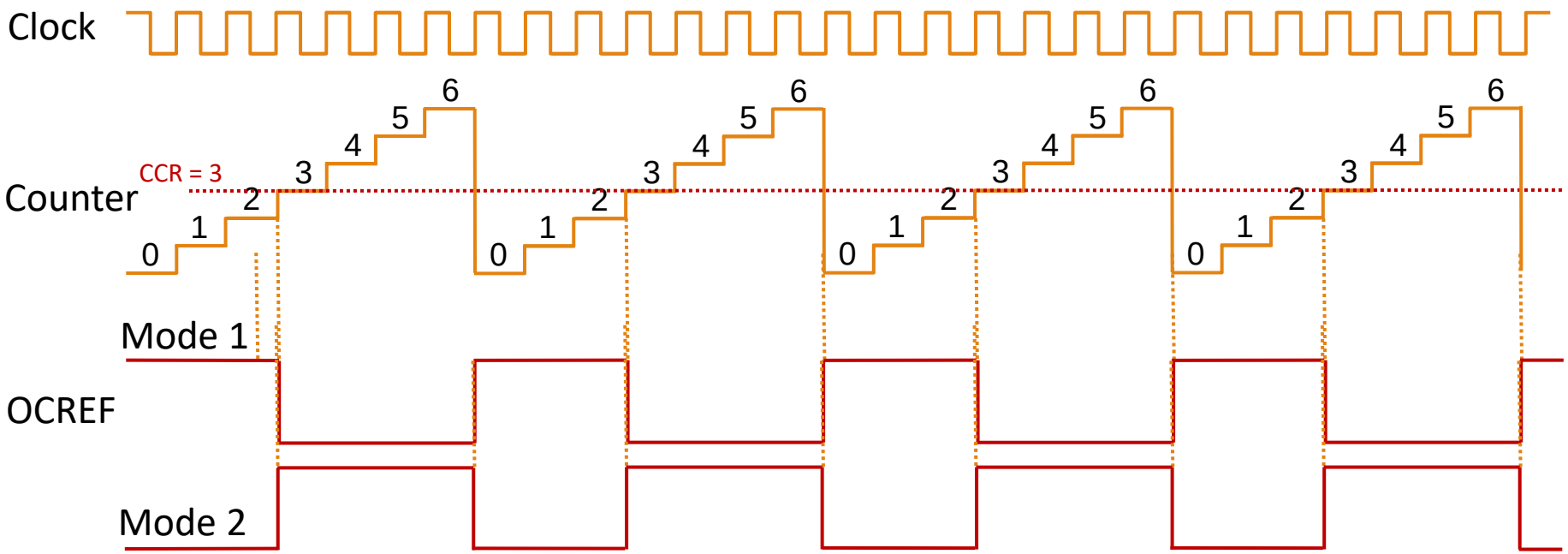
PWM Output setting:

1. Select Output mode as PWM	(**TIMx → CCMR : OCyM**)
2. Set CompareCapture value	(**TIMx → CCRy : CCR**)
3. Select Output Polarity	(**TIMx → CCER : CCyP**)
4. Enable CompareCaptureOutput	(**TIMx → CCER : CCyE**)

한동대학교 HANDONG GLOBAL UNIVERSITY

Timer Application : PWM

- Up-counting mode, ARR = 6, CCR = 3, PSC= 1



$$DutyCycle = \frac{CCR}{ARR + 1}$$

Mode1: 3/7

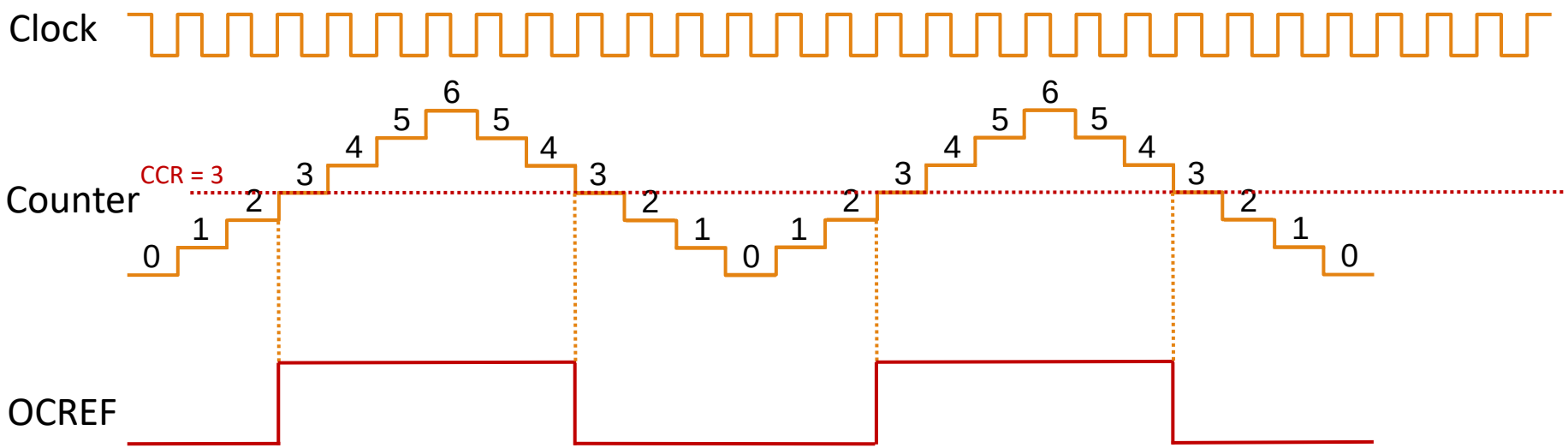
Mode2: 4/7

PWM Period = (1 + ARR) * CLK_Period
= 7 * CLK Period

한동대학교 HANDONG GLOBAL UNIVERSITY

Timer Application : PWM

- Center-aligned mode, ARR = 6, CCR = 3



$$\text{Duty Cycle} = 1 - \frac{CCR}{ARR} = \frac{1}{2}$$

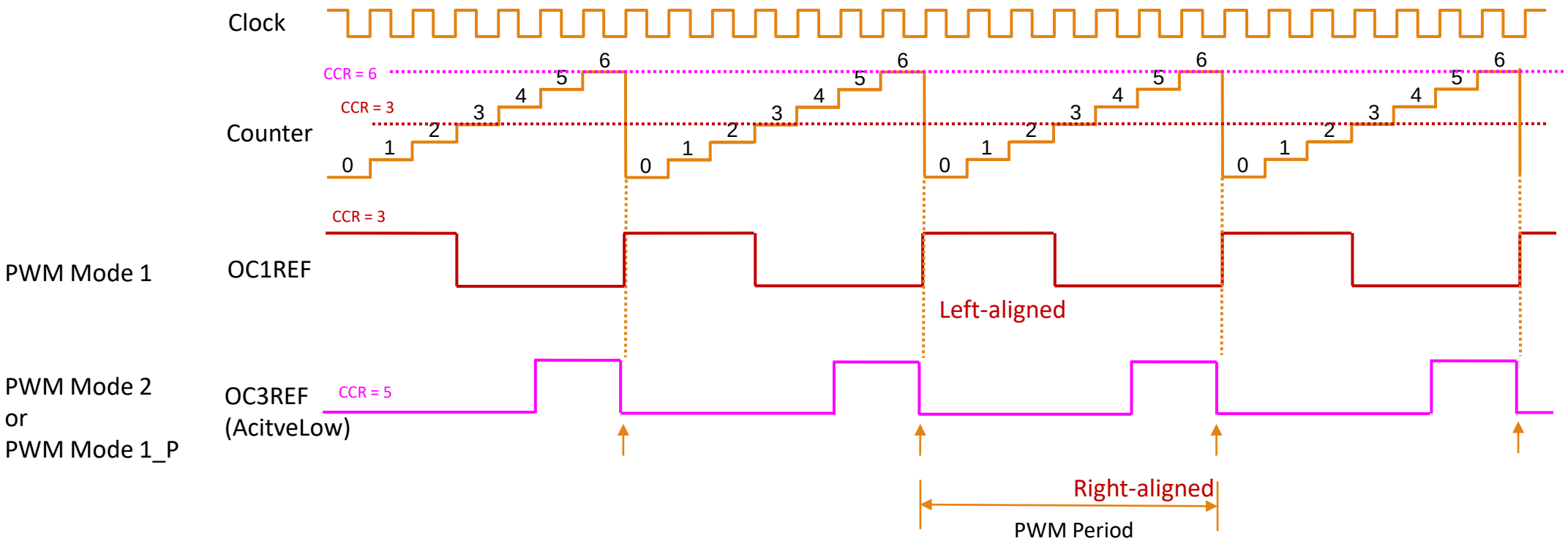
$$\text{Period} = 2 * ARR * \text{Clock Period} = 12 * \text{Clock Period}$$

한동대학교 HANDONG GLOBAL UNIVERSITY

PWM Alignment

- **Each channel can have different duty ratio with same period**
  - Up-Counter: PWM alignment Left or Right
  - Can choose polarity for each channel
  - TIMx_CCER→CCyP, y=1 to 4

Up-counting mode, ARR = 6, CCR = 3, RCR = 0

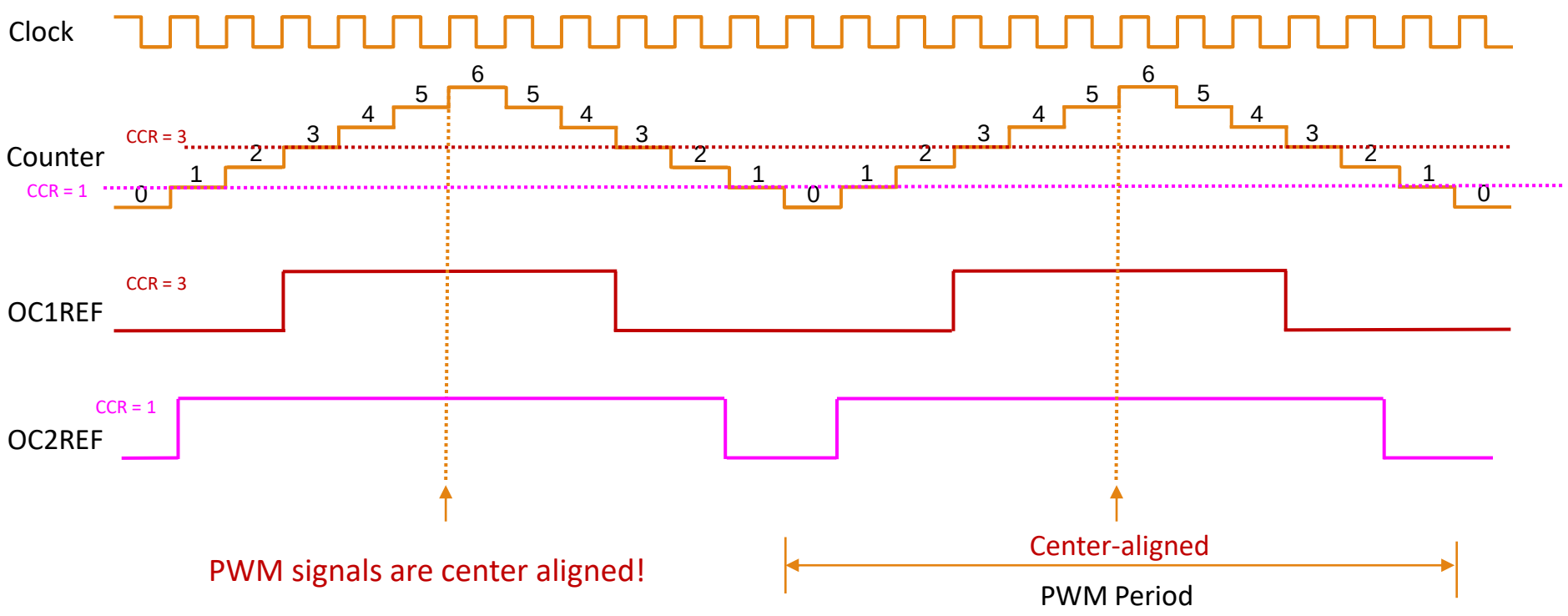


한동대학교 HANDONG GLOBAL UNIVERSITY

PWM Alignment

- **Each channel can have different duty ratio with same period**
  - Up-Down Counter: Center-aligned
  - Can choose polarity for each channel
  - TIMx_CCER→CCyP, y=1 to 4

Center Aligned Center-aligned mode, ARR = 6, CCR = 3



한동대학교 HANDONG GLOBAL UNIVERSITY

Timer Application : PWM

- **How to choose PWM period and duty ratio?**
  - PWM Period: PSC, ARR
  - Duty Ratio: CCR

- **Example:** Generate 1kHz PWM with duty ratio of 50% using 84MHz system clock

PWM frequency:

$F_{CK_CNT} = F_{CK_PSC} / (PSC+1)$

→ 1MHz= 84MHz/(83+1) // PSC=83

$F_{PWM} = F_{CK_CNT} / (1 + ARR)$

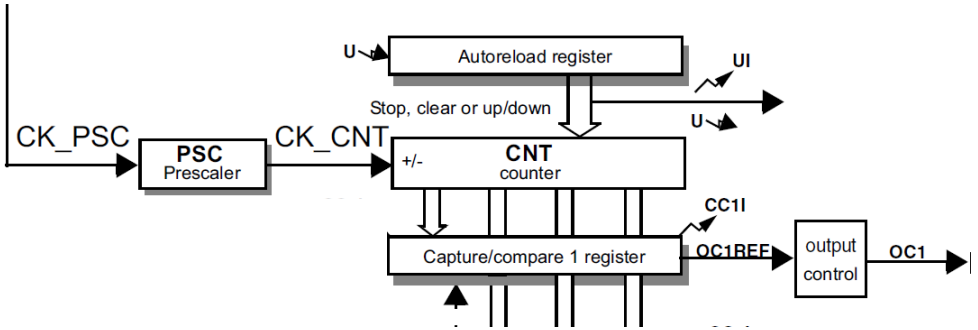
→ 1kHz= 1MHz/(1+999) // ARR=999

PWM duty ratio:

For mode 1:

DutyCycle=CCR/ARR+1

→ 0.5=500/(1+999) // CCR=500



Q) With PSC=83 for 84MHz system clock, what is the min/max of PWM frequency?

Note) For high resolution of PWM duty control, you need to have sufficient ARR numbers.
: If ARR=10, there is only 10 levels of PWM duty

한동대학교 HANDONG GLOBAL UNIVERSITY

Timer Application : PWM

- **How to choose PWM output pin?**
  - Can we choose two channels from the same timer for PWM output on two AF pins?
    - TIM1_CH1
    - TIM1_CH2

- **What is the maximum number of PWMs we can output simultaneously?**

Timer PinOut Map

Advanced Timer

Timer	Channel	Port	Pin
1	1	A	8
	1N	A	7
		B	13
	2	A	9
	2N	B	0
		B	14
	3	A	10
	3N	B	1
		B	15
	4 4N		

General Purpose Timer

Timer	Channel	Port	Pin
2	1	A	0
		A	5
		A	15
	2	A	1
		B	3
	3	B	10
	4		
3	1	A	6
		B	4
		C	6
	2	B	5
		C	7
	3	C	8
	4	C	9
4	1	B	6
	2	B	7
	3	B	8
	4	B	9

한동대학교 HANDONG GLOBAL UNIVERSITY

Appendix

Process	Sub-Process	Interrupt (Event)	Interrupt (CCR)	PWM
Clock	System CLK	RCC		
	TIM CLK Enable	RCC_APB1ENR		
Counter Setup	Period	PSC ARR		
	DIR	DIR		
	Enable	CEN		
Interrupt	Flag	UIF	CCxIF	
	Enable	UIE	CCxE	
Compare& Capture	Compare value	-	CCR	CCR
Output Setup	Pin	-		GPIO AF
	Mode	--	-	OCyM=110 OCyM=111
	Polarity	-		CCxP
	Enable	-		CCyE

한동대학교 HANDONG GLOBAL UNIVERSITY